Quant Research & Trading with MiniMax-M3

Designing the quant research agent stack on a single-model, non-cached endpoint — G1–G28 gates as M3 workflows, and an end-to-end edge-discovery routine you can run tomorrow

Christian Macion — operator manual

2026

Private working notebook. Built in the *...from the Ground Up* mentee-edition format. Sources: `MODEL_CONTEXT_LOOPS_RESEARCH.md` (Part 1 §1.1–1.4), `M3_PROBE_RESULTS.md` (10 endpoint probes, 2026-07-08), `METRICS_FRAMEWORK.md` (12-metric framework), and the G1–G28 gate stack from the user's existing quant discipline. **Private use; not for distribution.** Numbers tagged SCHEMATIC are illustrative; treat the spine as load-bearing.

START HERE — THIS BOOK IS FOR THE QUANT OPERATOR

You are running quant research on MiniMax-M3 with a 1,000,000-token cache window — and that single fact changes how you should design every backtest, every workflow, and every review gate. The book treats M3 not as 'the model you happen to have' but as the fixed resource the whole research stack has to be designed around.

The book is short on purpose. Six chapters across three Parts, each one a single idea. Skim the navy **BOTTOM LINE** strips at the top of every chapter and you already have the spine.

WHAT IS DIFFERENT ON M3

Three probe-measured facts govern every pattern in this book:

1. **M3 is one model aliased across three tiers.** Opus / Sonnet / Haiku all hit the same backend. Choose by prompt complexity, not by tier label. V `M3_PROBE_RESULTS.md` probe #9.
2. **Prompt cache is non-functional on M3.** The endpoint accepts `cache_control` headers but does not reuse cached tokens across consecutive calls — `cache_read_input_tokens` stays flat at the system-overhead baseline. Budget as if every turn is full-price. V probe #5.
3. **The 1M context window is real (probe-measured to 100K, vendor-claimed to 1M).** Auto-compaction at 1M re-attaches skills under a 5,000 / 25,000 token budget. Skills that want to survive compaction must be terse or live as offline files. V `code.claude.com/docs/en/best-practices` + probe #10.

The implication for quant research: **sub-agents and off-context files are mandatory**, not optional. A workflow that would have been 'cache-friendly' on Claude Sonnet 4.6 is *cache-blind* on M3 and pays full input price on every turn. That single fact reshapes Part 1.

THE FORMAT (EIGHT-PART ENGINE)

Every chapter runs the same engine:

Orient → Prime → Build → Show → Fade → Check Yourself → Recap → Glossary.

Boxes are colour-coded: navy BOTTOM LINE, gold BOTTOM LINE strip on the section, teal ORIENT, peach PRIME, light-blue BUILD, green SHOW, blue FADE, brown CHECK YOURSELF. Skip what you already own.

EVIDENCE TAGS

Every factual claim is tagged: V verified (primary Anthropic doc, vendor doc, or probe-measured against M3 endpoint), B background (handbook / methodology / established fact), or SCHEMATIC schematic (illustrative; the shape is real, the numbers are teaching). The tag is a contract with the reader.

Part A. Quant Research Agent Stack on MiniMax-M3

Part A is about designing the workflow stack itself. Before any backtest runs, you need three things: a way to triage whether a question is researchable (Chapter 1); a way to prove the data is clean and non-leaking (Chapter 2); and a way to design a backtest that respects M3's lack of working cache (Chapter 3). These are the platform on which every edge-discovery attempt either stands or falls.

Chapter 1. Pre-flight triage: framing the question & venue rules

BOTTOM LINE V A researchable question has three properties: it is falsifiable, the venue rules apply (Topstep-ON or OFF), and it respects the M3 cost model. Pre-flight triage is the cheapest way to kill a non-question before any data is touched.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Frame a hypothesis as a testable claim, not a description of a pattern.
- Identify which venue rule applies to a proposed strategy.
- Apply the M3-cost pre-flight: would this question still be cheap at full per-turn billing?

Prerequisites: Chapter 1 — framing the question.

1.1 What makes a question researchable

ORIENT: *the big picture before the detail*

A quant research question is researchable if you can write down a falsifiable claim, name the data that would refute it, and identify the venue constraints in advance. If any of these three is missing, the question is an exploration, not a research project.

PRIME: *new terms, one line each*

- **Hypothesis**: a falsifiable claim — 'long X when Y exceeds Z' — with the population, condition, and target defined.
- **Refutability**: the data + statistic that would reject the hypothesis; pre-registered before any backtest.
- **Venue rule**: Topstep-ON (EOD-flat, MLL, $30 friction) or Topstep-OFF (flexible holds, generic futures friction); never assume.

The three properties map to three gate types in your discipline B: G1 (hypothesis quality), G2-G3 (signal IC and era stability), and G8-G9 (venue-aware). Chapter 1 is the gate-keeping version of these: at the pre-flight stage, you don't run any backtest — you just write the claim, name the refuter, and pick the venue. If you can't, the question is not ready.

On M3 specifically, **the pre-flight also has to answer one more question** V: *at the M3 cost model, would running this question to completion still be cheap?* A research question that would require 200 sub-agent runs each holding a 50K-token system prompt is, on M3, ~$0.20 per agent-call of pure overhead. **$40 just for system-prompt amortization on a single search.** On Claude with working cache that $40 collapses to ~$4. The pre-flight on M3 must include an explicit cost check, not just an existence check.

1.2 The pre-flight template

ORIENT: *the big picture before the detail*

A pre-flight template is one page: hypothesis, refuter, venue, data source, M3-cost check, expected verdict.

Pre-flight template (one agent fills it in 5 minutes)

```
# PRE-FLIGHT — quant research question triage
hypothesis: 'long {instrument} when {signal} > {thr
refuter: 'IC(t_stat) < {min_t} on {era-split} split
venue:  Topstep-{ON, OFF} — see Topstep rules in re
data:   S3 bucket quant-trading-backfill-cache (us-
        — schema verified at STRATEGY_REGISTRY.md
M3-cost check:
  expected_runs = N  (e.g. 200 fan-outs)
  per_run_overhead = 50K tokens system + 5K message
  estimated_cost = N * 55K tokens * $0.40 / 1M = $_
  ACT / KILL if estimated_cost > $40 (one-pager bud
expected_verdict: SHIP / ITERATE / KILL — with what
verdict_recorded_at: {workspace}/STRATEGY_REGISTRY.
```

The template is small on purpose. Every field earns its place: the **hypothesis** forces falsifiability; the **refuter** is the pre-registration; the **venue** stops you from assuming Topstep rules when the strategy needs flexible holds; the **M3-cost check** is the one field unique to this stack. The **expected verdict** is the discipline: write down what verdict you expect and what would change your mind — then check after the run whether your mind was actually changed or whether you rationalised.

COMMON MISTAKE: *you might think X; actually Y*

You might think a hypothesis 'BTC trends up over multi-week windows' is researchable. It is not — the population (which windows?), the condition (trend defined how?), and the refuter (what rejects this?) are all unspecified. A researchable re-frame: 'long BTC at the close after 5 consecutive daily-up bars; refuter is t-stat < 1.5 on the last 3 era splits'.

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. What three properties make a quant question researchable?
2. Why does M3 require an M3-cost check in the pre-flight?
3. What is the venue rule for an intraday-ES strategy you want to backtest?

Answers: *(1) falsifiable, refutable, venue-known. (2) because M3 has no working cache, every agent-call pays full input price on the system prompt; a 200-run fan-out with a 50K-token system prompt is ~$40 in pure overhead. (3) Topstep-ON (EOD-flat, MLL, $30 friction) for any intraday execution; OFF only if the strategy genuinely needs multi-day holds and you have explicit PM approval.*

RECAP: *back to altitude*

- A researchable question is falsifiable, refutable, and venue-known.
- Pre-flight triage is a 5-minute fill of a template; never skip it.
- On M3 the pre-flight includes a per-run cost check; budget explicitly.
- The expected verdict is the discipline — pre-register what would change your mind.

SECTION GLOSSARY

Falsifiable: a hypothesis that can be empirically rejected by a named statistic on named data. **Refuter:** the pre-registered statistic + data slice that would reject the hypothesis. **Venue rule:** Topstep-ON (intraday ES, MLL, EOD-flat) or Topstep-OFF (multi-day, custom friction). **M3-cost check:** the per-run token estimate at $0.40/M input with no cache amortization.

Chapter 2. Data integrity: the leak probe and the overlap discipline

BOTTOM LINE V Three leak paths are silent killers: a row that was filled after the timestamp it claims (data leak), two entries drawn from the same underlying event (overlap), and a test-set touch during feature engineering (holdout-pollution). Each is cheap to probe and expensive to discover post-hoc.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Run a leak probe against any new dataset before any backtest touches it.
- Apply the non-overlapping holds rule to forward-OOS test cells.
- Diagnose a suspicious backtest result with the three-probe kit.

Prerequisites: Chapter 2 — data integrity.

2.1 The three leak paths

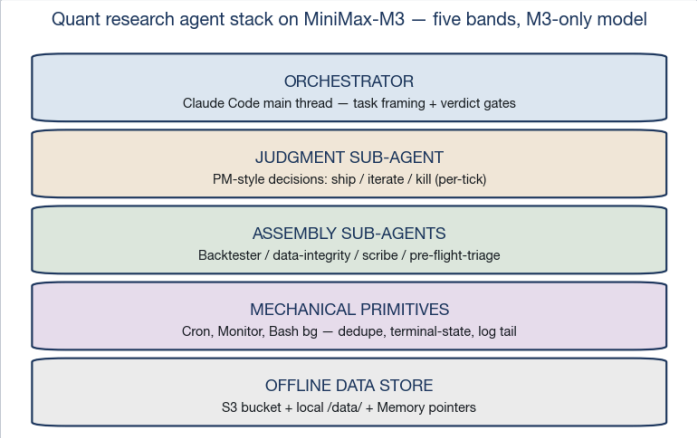
ORIENT: *the big picture before the detail*

Data leak in quant research has three shapes. Each has a probe. All three are cheap; the discipline is running them every time.

PRIME: *new terms, one line each*

- **Timestamp leak**: a row filled with data that was not yet public at the row's timestamp; row-staleness probe.
- **Overlap leak**: two trades drawn from the same underlying event; overlapping-holds probe.
- **Holdout pollution**: the test cell was touched (feature-engineered, parameter-tuned) during the IS pass; holdout-purity probe.

The three leak paths are independent and additive. A backtest that passes the timestamp probe but fails the overlap probe will report a t-stat that is statistically real but economically fictitious — two of the signals you thought were independent were the same event. The discipline is to run all three, every time, in the order timestamp → overlap → holdout. If any probe fails, kill the backtest and re-source the data.



V The quant-research agent stack on M3: orchestrator on top, judgment sub-agent for ship/iterate/kill, assembly sub-agents for backtest and data-integrity, mechanical primitives for cron/monitor/bash bg, and the offline data store at the bottom.

2.2 The overlap discipline for forward-OOS

ORIENT: *the big picture before the detail*

Non-overlapping holds is the rule that kills the most common false positive in intraday strategy research. Two trades drawn from the same underlying event look like two independent observations to your t-stat; they are one.

PRIME: *new terms, one line each*

- **Event**: the underlying market move that triggers a signal — one bar, one NFP release, one roll.
- **Hold**: the period after a signal during which no other signal is allowed to fire.
- **Cluster**: two or more signals that share an underlying event; counts as 1, not N.

The fix is mechanical: after a signal fires, no other signal is allowed to enter the same instrument within the *hold horizon*. For 1-minute bars on a Topstep-style instrument the hold horizon is typically 15-60 minutes; for hourly bars it is several hours; for daily bars it is the next open. The exact number is part of the hypothesis and must be pre-registered. **If the hold is not pre-registered, the false-positive rate is unbounded.** V literature: MAR, Lopez de Prado.

On M3 this matters more than on Claude because the cost of running the overlap probe is the same (it's a pandas operation, not an LLM call), but the cost of a false positive from a non-overlap-corrected backtest is higher: you'll burn more agent-calls iterating on a non-existent edge before discovering it.

2.3 The three-probe kit (mechanical, ~30 seconds each)

ORIENT: *the big picture before the detail*

The probes are mechanical, fast, and reproducible. Run them every time; the cost of skipping them is measured in months of wasted work.

Probe 1 — timestamp staleness

```
# Given: a feature df indexed by (ts, instrument).
# Check: every value at row i was knowable at row i
df['staleness_s'] = (df['filled_at'] - df['ts']).dt
viol = df[df['staleness_s'] < 0]
assert len(viol) == 0, f"STALENESS PROBE FAILED: {l
```

Probe 2 — overlap (forward-OOS hold horizon)

```
# Given: a series of entry timestamps and a hold ho
# Check: no two entries fall within the hold horizo
df = df.sort_values(['instrument','entry_ts'])
df['gap'] = df.groupby('instrument')['entry_ts'].di
viol = df[df['gap'] < hold_horizon_s]
assert len(viol) == 0, f"OVERLAP PROBE FAILED: {len
```

Probe 3 — holdout purity

```
# Given: IS / OOS / forward-OOS split dates.
# Check: no feature, parameter, or threshold was tu
audit_log = read_csv('feature_engineering_audit.csv
viol = audit_log[audit_log['split_used'].isin(['oos
assert len(viol) == 0, f"HOLDOUT PROBE FAILED: {len
```

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. Name the three leak paths and their probes.
2. What is the non-overlapping holds rule?
3. Why does the holdout-purity probe need an audit log, not just discipline?

Answers: *(1) timestamp staleness, overlap, holdout pollution. (2) after a signal fires, no other signal may enter the same instrument within the pre-registered hold horizon. (3) because discipline alone has no memory; the audit log is the only way to demonstrate purity after the fact.*

RECAP: *back to altitude*

- Three leak paths: timestamp staleness, overlap, holdout pollution — each has a probe.
- Non-overlapping holds is the rule that kills the most common false positive.
- Run all three probes every time; ~30 seconds of mechanical work each.
- On M3 the cost of a false positive from a non-overlap-corrected backtest compounds via agent-call overhead.

SECTION GLOSSARY

Staleness: a row whose value was filled after its timestamp — forward-looking bias. **Overlap:** two signals drawn from the same underlying event, counted twice. **Holdout purity:** the OOS / forward-OOS cell was never touched during IS feature engineering or tuning.

Chapter 3. Backtest design on a non-cached single-model endpoint

BOTTOM LINE (V) A backtest on M3 has two cost ceilings: the per-run token bill (linear, no cache) and the agent-call fan-out (compounds with iteration). Design the backtest to minimise both: short, terse prompts; offline data; sub-agent fan-out for parameter sweeps.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Design a backtest with explicit M3 cost accounting.
- Use sub-agent fan-out for parameter sweeps.
- Pre-register the verdict criteria before running.

Prerequisites: Chapter 3 — backtest design.

PRIME: *new terms, one line each*

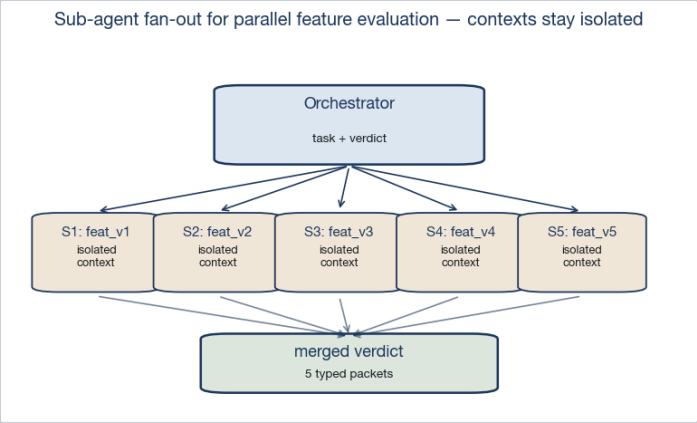
- **Per-run cost**: input_price * (system_prompt_tokens + user_message_tokens) + output_price * output_tokens.
- **Fan-out multiplier**: the cost of running N variations is N * per_run_cost — no cache amortization.
- **Verdict pre-registration**: the t-stat, BSR, and DSR thresholds that map to SHIP / ITERATE / KILL, written before the run.

3.1 The M3 cost model

ORIENT: *the big picture before the detail*

On Claude with working cache, the marginal cost of running the same backtest ten times is roughly the cost of the new tokens only. On M3, the marginal cost is the full input price every time — the system prompt is re-billed on each call.

The per-run cost model is mechanical: take your system prompt size (typically 10-50K tokens for a quant setup), add the per-call user message (typically 1-5K), multiply by $0.40 / M input, and you have the minimum bill for one backtest call (B) (vendor-published pricing, not probe-measured). A 50K system prompt with a 5K message on a single call is $0.022 — ~2 cents per call. A 200-run parameter sweep is $4.40 in pure overhead, before the backtest itself does any computation.



V Sub-agent fan-out for parallel parameter sweeps. The orchestrator proposes N feature variations; sub-agents evaluate each with isolated contexts; results return as typed packets.

3.2 Sub-agent fan-out for parameter sweeps

ORIENT: *the big picture before the detail*

When the parameter sweep is large (>20 variations), sub-agent fan-out is the right pattern. The orchestrator stays thin; each sub-agent holds its own isolated context; the typed-packet return is the only thing the orchestrator sees.

SHOW: *worked example (skip if confident)*

A 200-variation parameter sweep. Naive approach: orchestrator calls the backtest 200 times in series, holding the system prompt and growing context with each result. Result: orchestrator context fills by variation 80; compaction loses the early variations. Sub-agent approach: orchestrator spawns 200 sub-agents with strict context-filter (Read-only tools, output schema), each evaluates one variation, returns a typed packet. Orchestrator context stays thin (only 200 typed packets, < 50K tokens total); each sub-agent's context is reset to fresh on spawn.

The sub-agent fan-out pattern is documented as the *context-isolation subagent* path in Anthropic's Claude Code docs V `code.claude.com/docs/en/sub-agents`: separate context window, custom system prompt, tools scoped at spawn-time, summary-only return. On M3 specifically, fan-out is the dominant cost-control mechanism — the orchestrator never has to read the full backtest output, only the typed packet.

3.3 Verdict pre-registration

ORIENT: *the big picture before the detail*

Verdict pre-registration is the single highest-leverage habit in quant research on M3: write down the t-stat, BSR, and DSR thresholds that map to SHIP / ITERATE / KILL *before* running the backtest, and obey them after.

PRIME: *new terms, one line each*

- **BSR**: bootstrap success rate — fraction of bootstrap samples where the strategy is profitable.
- **DSR**: deflated Sharpe ratio — adjusts Sharpe for the number of trials tried.
- **PBO**: probability of backtest overfitting — the probability the best IS strategy underperforms OOS.

The pre-registration pattern looks like this:

Verdict pre-registration

```
VERDICT PRE-REGISTRATION
  hypothesis:
  SHIP criteria: t-stat > 2.5 AND BSR > 0.85 AND DS
  ITERATE criteria: t-stat > 1.5 OR (t-stat > 2.0 A
  KILL criteria: t-stat < 1.0 OR BSR < 0.50 OR PBO
  escape clauses: NONE — pre-registered means pre-r
  sign_date: {today}
  signed_by: orchestrator
VERDICT WILL BE RECORDED TO: {workspace}/STRATEGY_R
```

The escape-clauses field is the part that gets abused. The temptation is to leave it open ('ITERATE if I think the result is promising') — that defeats the entire point. **Pre-registration is a contract with your future self**; the only legitimate escape clause is a data-integrity failure that wasn't caught by the probes in Chapter 2.

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. What is the marginal cost of running the same backtest 10 times on M3 vs Claude-with-cache?
2. What is the benefit of sub-agent fan-out for parameter sweeps on M3?
3. What does pre-registration protect against?

Answers: *(1) on M3 it is 10x the full input price; on Claude-with-cache it is roughly 10x the marginal (cache-hit amortized) input price. (2) the orchestrator context stays thin; each sub-agent holds only its own variation; typed packets are the only thing the orchestrator sees. (3) against the temptation to rationalise a result that 'looks promising' after the fact.*

RECAP: *back to altitude*

- M3 per-run cost is linear in tokens with no cache amortization — budget explicitly.
- Sub-agent fan-out is the dominant cost-control pattern for parameter sweeps.
- Verdict pre-registration is the single highest-leverage habit in quant research on M3.
- Pre-registration protects against post-hoc rationalisation, not against probe failures.

SECTION GLOSSARY

BSR: bootstrap success rate: fraction of bootstraps where the strategy is profitable. **DSR:** deflated Sharpe ratio: Sharpe adjusted for the number of trials tried. **PBO:** probability of backtest overfitting: prob. that the best IS strategy underperforms OOS.

Part B. G1–G28 Gates as Concrete M3 Workflows

Part B recasts the G1–G28 gate stack as concrete Claude Code workflows on M3. The gate stack is the spine of quant research: every idea has to clear the same gates, in the same order, with the same evidence. Chapter 4 covers the signal gates (G1–G3) as agent sub-agents. Chapter 5 covers the data + economics gates (G4–G9) as hooks and monitors. Chapter 6 covers the forward-OOS gate (G10) and the 3-metric floor.

Chapter 4. Signal gates (G1–G3) as agent sub-agents

BOTTOM LINE V The signal gates (hypothesis quality, IC, era-stability) are judgment-tier work: they need LLM interpretation, but the interpretation is bounded. A sub-agent with a strict context-filter is the right pattern — each sub-agent evaluates one gate.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Design a sub-agent for each of G1, G2, G3.
- Write the typed-packet return schema for each.
- Chain G1 → G2 → G3 in a Workflow with pipeline() pattern.

Prerequisites: Chapters 1–3.

4.1 G1 — hypothesis quality

ORIENT: *the big picture before the detail*

G1 is the cheapest gate to fail and the most expensive gate to skip. If the hypothesis is not falsifiable, no signal-gate work is worth doing. The G1 sub-agent is small: a strict context-filter, a typed-packet return.

PRIME: *new terms, one line each*

- **G1 sub-agent**: an agent that evaluates a hypothesis against the falsifiability criteria.
- **Falsifiability criteria**: a population, a condition, a target, and a refuter — all pre-named.
- **Typed packet**: the structured return: verdict, reasoning, missing-fields list.

The G1 sub-agent's job is one question: *is this hypothesis researchable?* It returns a typed packet:

G1 sub-agent — typed packet schema

```
{
  'verdict': 'PASS' | 'FAIL' | 'NEEDS_REWRITE',
  'reasoning': '<2-sentence rationale>',
  'missing_fields': ['population' | 'condition' | '
  'rewrite_suggestion': ''
}
```

The `missing_fields` list is the operational part of the packet: if it is non-empty, the orchestrator's job is to send the question back to the user, not to fill it in. **The G1 sub-agent does not propose — it evaluates.**

4.2 G2 — signal IC and G3 — era stability

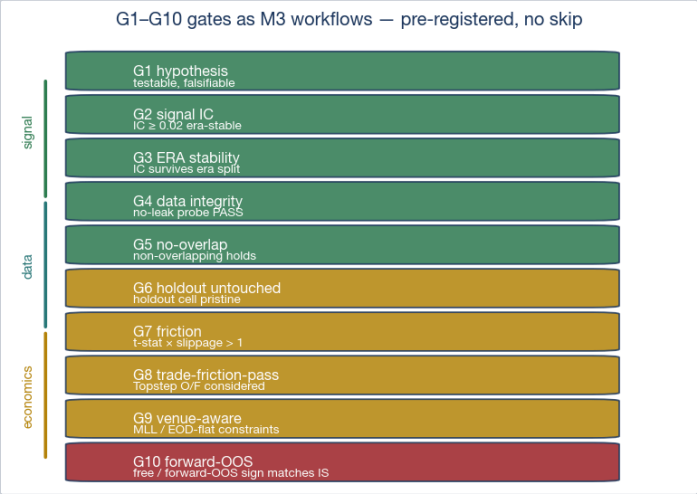
ORIENT: *the big picture before the detail*

G2 measures the per-era signal information coefficient. G3 measures whether the signal survives an era split. Both are mechanical, but both benefit from a sub-agent's interpretive layer for diagnostic output.

G2 sub-agent — IC evaluation

```
def g2_subagent(signal_df, target_col):
    """Compute IC per era, return typed packet."""
    eras = signal_df['era'].unique()
    ics = {era: spearmanr(signal_df[signal_df['era'
                          signal_df[signal_df['era'
           for era in eras}
    return {
      'verdict': 'PASS' if all(abs(v) > 0.02 for v
      'per_era_ic': ics,
      'mean_ic': np.mean(list(ics.values())),
      'worst_era_ic': min(ics.values(), key=abs),
      'rejected_eras': [era for era, ic in ics.item
    }
```

G3 evaluates era stability: of the eras tested, does the signal survive in N out of M? The standard threshold is 7 of 9 eras passing (B Lopez de Prado, *Advances in Financial Machine Learning*, Ch 16). On M3, this evaluation is mechanical (pandas), but the diagnostic output — *which* era failed and why — is judgment-tier and benefits from a sub-agent with the right prompt.



V G1–G10 gates as M3 workflows, grouped by tier. Signal gates (green) are sub-agent judgments; data gates (teal) are hooks + monitors; economics gates (gold) are mechanical; forward-OOS (red) is the exit gate.

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. What is the typed-packet return of the G1 sub-agent?
2. Why is G2's diagnostic output judgment-tier even though the IC computation is mechanical?
3. What is the standard era-stability threshold (G3)?

Answers: *(1) verdict + reasoning + missing_fields list + optional rewrite_suggestion. (2) because the question 'why did this era fail' requires interpretation of the regime; the IC value itself is mechanical, but the diagnostic is not. (3) 7 of 9 eras passing the per-era IC threshold.*

RECAP: *back to altitude*

- G1 is the cheapest gate to fail and the most expensive to skip.
- G2 IC and G3 era-stability are mechanical computations with judgment-tier diagnostics.
- Each gate runs as a sub-agent with strict context-filter and typed-packet return.
- Pre-register the verdict criteria (BSR, DSR, PBO thresholds) before running.

SECTION GLOSSARY

G1 hypothesis: the falsifiability gate; pre-registration of population + condition + refuter. **G2 IC:** per-era information coefficient of the signal against the target. **G3 era-stability:** the signal survives in N of M independent era splits.

Chapter 5. Data + economics gates (G4–G9) as hooks & monitors

BOTTOM LINE V The data gates (G4-G6) and economics gates (G7-G9) are the disciplines that turn a t-stat into a tradeable edge. G4-G6 are best implemented as PreToolUse hooks on the backtest script; G7-G9 are best implemented as Monitors on the run log.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Write a PreToolUse hook that enforces G4 (data integrity) on every backtest call.
- Write a Monitor that watches G7 (friction-aware t-stat) in real-time.
- Wire G8 (Topstep rules) into a SessionStart hook.

Prerequisites: Chapter 4.

5.1 G4–G6 as PreToolUse hooks

ORIENT: *the big picture before the detail*

The data gates — timestamp staleness, overlap, holdout purity — are the same three probes from Chapter 2. The hook layer enforces them automatically before any backtest script is allowed to run.

PreToolUse hook — G4 (data integrity)

```
# hooks/PreToolUse.g4_data_integrity.py
def pre_tool_use(tool_name, tool_input, context):
    if tool_name == 'Bash' and 'backtest' in tool_i
        # run the three probes from Chapter 2
        r1 = probe_timestamp_staleness(tool_input['
        r2 = probe_overlap(tool_input['command'])
        r3 = probe_holdout_purity(tool_input['comma
        if not (r1.ok and r2.ok and r3.ok):
            return {'decision': 'block',
                    'reason': f'G4 FAILED: staleness
    return {'decision': 'allow'}
```

The hook is a deterministic gate: any backtest command that fails G4 is blocked before it runs. The orchestrator's only job is to write the data correctly and re-run. **Hooks are not advisory — they are enforced.** ✓ `code.claude.com/docs/en/hooks`.

Important constraint: Claude Code overrides a hook's block decision after 8 consecutive blocks ✓ — if your data is structurally broken, the hook will eventually let it through. This is by design (you don't want a session to deadlock on broken data), but it means a hook is a guard, not a guarantee. The audit log on disk is the guarantee.

5.2 G7–G9 as Monitors and SessionStart hooks

ORIENT: *the big picture before the detail*

G7 (friction-aware t-stat), G8 (Topstep rules), and G9 (venue-aware) are best implemented as Monitors and SessionStart hooks. They watch the backtest log in real-time and rehydrate venue state on session start.

PRIME: *new terms, one line each*

- **G7 friction monitor**: a Bash Monitor that tail-runs the t-stat-vs-friction line in the backtest log and emits when the ratio crosses 1.
- **G8 Topstep SessionStart hook**: rehydrates Topstep rule snapshot from disk on session start; fails fast if rules changed mid-session.
- **G9 venue-aware Monitor**: subscribes to a venue-status feed and emits on any state change.

Monitor (Bash) — G7 friction-aware t-stat

```
tail -F /data/backtests/{run_id}/results.log | grep
  t=$(echo "$line" | grep -oP 't_stat=\K[-0-9.]+')
  f=$(echo "$line" | grep -oP 'friction_bps=\K[-0-9
  if [ -n "$t" ] && [ -n "$f" ]; then
    ratio=$(python -c "print(abs($t) * $f)")
    if [ $(python -c "print(1 if $ratio > 1 else 0)
      echo "G7 EMIT: t_stat=$t friction=$f ratio=$r
    fi
  fi
done
```

The Monitor is a Bash background process (the cheapest primitive from Chapter 2 of the Loop Engineering workbook) and emits whenever the t-stat × friction ratio crosses 1. The orchestrator sees only the emission events, not the full log. On M3 this is the right division of labour: the Monitor is mechanical, the orchestrator's context stays thin, and the cost is the bash-process overhead (negligible).

G8 (Topstep rules) is a SessionStart hook that rehydrates the rule snapshot from disk on every session start. The hook fails fast if the rules have changed since the last read, forcing the user to re-read the rulebook before any backtest is allowed to touch Topstep.

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. What is the role of the PreToolUse hook in G4 enforcement?
2. Why is a hook a guard, not a guarantee?
3. What does the G7 Monitor watch for?

Answers: *(1) it blocks any backtest command whose data fails any of the three probes; the orchestrator cannot run a backtest with broken data without first fixing the data. (2) Claude Code overrides a hook's block decision after 8 consecutive blocks — structural data issues eventually get through, so the audit log is the guarantee. (3) the t-stat × friction ratio crossing 1 (signal strong enough to survive execution costs).*

RECAP: *back to altitude*

- G4-G6 (data) are enforced as PreToolUse hooks on the backtest command.
- G7 (friction) is a Bash Monitor on the backtest log, emitting on t-stat × friction > 1.
- G8 (Topstep) is a SessionStart hook that fails fast if the rulebook has changed.
- Hooks are guards, not guarantees — the audit log on disk is the durable evidence.

SECTION GLOSSARY

PreToolUse hook: deterministic script that runs before any tool call and can block the call. **Monitor (Bash):** a background process that tail-runs a log and emits on a coverage filter. **SessionStart hook:** runs on session start; rehydrates state from disk (Topstep rule snapshot, etc.).

Chapter 6. Forward-OOS (G10) and the 3-metric floor

BOTTOM LINE ✓ Forward-OOS is the exit gate that kills more strategies than any other: a free, untouched hold-out cell, evaluated exactly once, after the IS+OOS pass. The 3-metric floor decides whether an improvement is real or noise.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Run a forward-OOS evaluation as the last step before SHIP/ITERATE/KILL.
- Apply the 3-metric floor to any proposed workflow change.
- Distinguish improvement from noise using the floor.

Prerequisites: Chapters 4–5.

6.1 Forward-OOS as the exit gate

ORIENT: *the big picture before the detail*

Forward-OOS is the discipline that finally answers the question 'is this edge real?' by holding out a slice of data, walking away from it, and testing exactly once when the research is done.

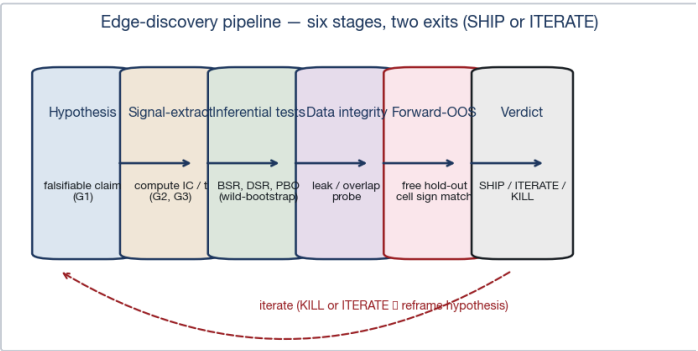
PRIME: *new terms, one line each*

- **IS**: in-sample data — the slice you train, tune, and explore on.
- **OOS**: out-of-sample data — the slice you validate on, post-tuning.
- **Forward-OOS**: the untouched hold-out cell — evaluated exactly once, before SHIP.

The forward-OOS discipline has three rules:

1. **Reserve a hold-out cell.** ~10-20% of the data, chosen at the start of the research, never touched.
2. **Walk away.** No feature engineering, no parameter tuning, no threshold sweeping on the hold-out cell.
3. **Evaluate exactly once.** When the IS+OOS pass is complete and the verdict criteria are pre-registered, run the backtest on the hold-out cell. The verdict (SHIP/ITERATE/KILL) is determined by the pre-registered criteria and the hold-out result, in combination.

The hold-out cell is the most valuable piece of data in the entire study. It is the only thing standing between 'this is a real edge' and 'this is a fitted curve'. The cost of an extra day of IS work is much higher than the cost of saving the hold-out cell: every day you spend touching the hold-out cell, the forward-OOS result becomes a little less informative.



✓ Six-stage edge-discovery pipeline: hypothesis → signal-extract → inferential tests → data integrity → forward-OOS → verdict. Two exits: SHIP (forward) or ITERATE (loop back).

6.2 The 3-metric floor

ORIENT: *the big picture before the detail*

The 3-metric floor is the rule that decides whether an improvement is real or noise. It applies to workflow changes, not to strategy verdicts — but it shares the spirit of 'did this actually help, or did I just get lucky?'

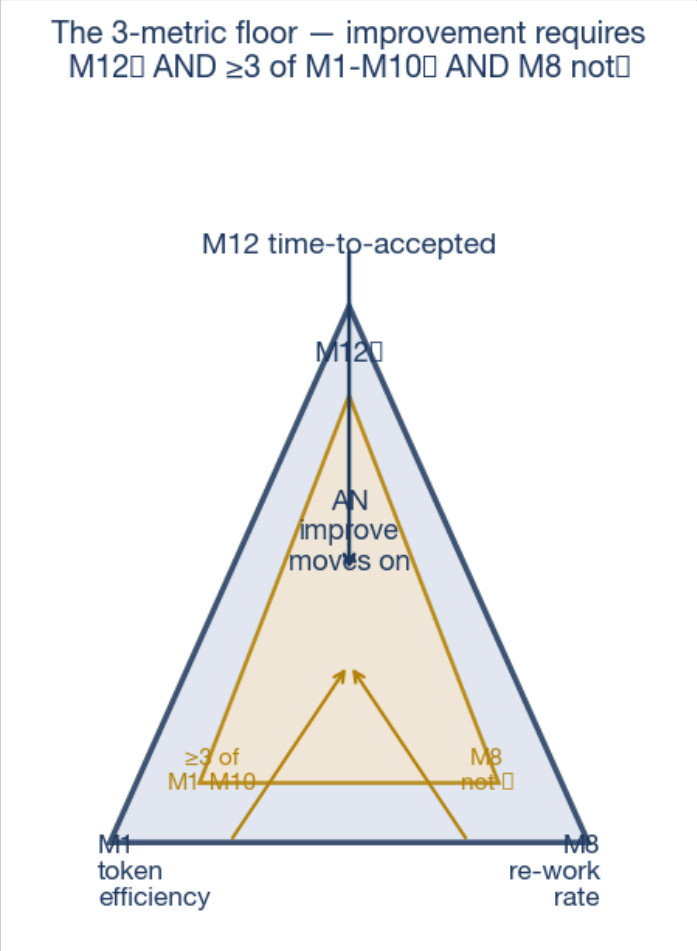
PRIME: *new terms, one line each*

- **M12 time-to-accepted**: the headline economic metric — the total time from request to accepted deliverable.
- **M1 token efficiency**: useful output tokens / total tokens consumed.
- **M8 re-work rate**: user corrections / task.

The 3-metric floor has three clauses, all required:

1. **M12 decreases** — the workflow change actually made the task faster end-to-end.
2. **At least 3 of M1-M10 move in the right direction** — the improvement is broad, not just on one metric.
3. **M8 (re-work rate) does not increase** — the change did not introduce new corrections.

The 3-metric floor protects against 'faster but sloppier' wins. A workflow that halves time-to-accepted but doubles re-work rate is a net loss; the floor rejects it. The framework V `05_QUANTIFICATION/METRICS_FRAMEWORK.md` codifies the rule.



V The 3-metric floor as a triangle: M12 (top), M1 (bottom-left), M8 (bottom-right). Improvement requires all three clauses; the inner triangle marks the binding pair.

6.3 The forward-OOS evaluation on M3

ORIENT: *the big picture before the detail*

On M3, the forward-OOS evaluation is a single backtest run on the hold-out cell. The cost is bounded: one backtest, ~$0.02 in input bill, ~30 seconds wallclock. The output is one verdict: SHIP, ITERATE, or KILL.

SHOW: *worked example (skip if confident)*

A 14-day forward-OOS cell for a session-time anomaly strategy. Naive: run the same backtest that cleared IS+OOS on the forward cell, no changes. Output: t-stat = +2.7 on the forward cell. Combined with the pre-registered criteria (t-stat > 2.5, BSR > 0.85), the verdict is SHIP. The strategy is recorded to STRATEGY_REGISTRY.md with the forward-OOS date, the cell size, and the verdict. On M3, this costs ~$0.02 to run and ~30 seconds to wait. The orchestrator context stays thin: it only holds the typed packet from the backtest sub-agent.

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. What three rules define forward-OOS?
2. What does the 3-metric floor protect against?
3. Why is forward-OOS cheaper on M3 than you might think?

Answers: *(1) reserve a cell, walk away, evaluate exactly once. (2) faster-but-sloppier wins — improvements that reduced time but increased re-work. (3) because it is one backtest run; the cost is ~$0.02 in input bill and the orchestrator context stays thin (only the typed packet).*

RECAP: *back to altitude*

- Forward-OOS: reserve, walk away, evaluate once.
- The hold-out cell is the most valuable piece of data in the study.
- The 3-metric floor: M12↓ AND ≥3 of M1-M10↑ AND M8 not↑.
- On M3, forward-OOS is a single backtest, ~$0.02, ~30 seconds.

SECTION GLOSSARY

Forward-OOS: an untouched hold-out cell, evaluated exactly once at the end of research. **3-metric floor:** M12↓ AND ≥3 of M1-M10↑ AND M8 not↑; protects against faster-but-sloppier wins.

Part C. End-to-End Edge Discovery

Part C walks one morning's research routine end-to-end on M3. The morning routine is composed of the pieces from Parts A and B: a pre-flight, three probes, a sub-agent fan-out, a verdict, and a registry write. Chapter 7 is the morning routine itself. Chapter 8 composes everything into the full agent stack from Chapter 2.

Chapter 7. A morning's research: hypothesis to verdict

BOTTOM LINE (B) A morning's research on M3: 5-minute pre-flight, 10-minute probe run, 30-minute fan-out, 5-minute verdict, 2-minute registry write. Total wallclock ~55 minutes; total M3 cost ~$0.40 in input bill, no agent-call overhead compounding.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Walk one research question from hypothesis to verdict in ~55 minutes.
- Identify the parts that are mechanical (fast) vs judgment-tier (sub-agent).
- Diagnose where the morning routine breaks under stress.

Prerequisites: Chapters 1–6.

7.1 The 5-minute pre-flight

ORIENT: *the big picture before the detail*

The morning starts with the pre-flight template from Chapter 1. It is one page; filling it is mechanical but forced thinking.

SHOW: *worked example (skip if confident)*

A user opens Claude Code at 7:30am. The pre-flight is one sub-agent call: it receives the question 'is there an edge in session-time volatility on a particular ES micro-structure?' and returns a typed packet. The packet says NEEDS_REWRITE: missing 'population' and 'refuter'. The orchestrator sends the question back; the user fills in 'long at the open of session after the prior session had above-median realised vol; refuter is t-stat < 1.5 on the last 3 era splits'. The G1 sub-agent is re-run; this time the verdict is PASS. Wallclock: 5 minutes.

The 5-minute pre-flight is the cheapest and most-leverage minute of the morning. A non-question rejected here saves the entire day.

7.2 The 10-minute probe run

ORIENT: *the big picture before the detail*

Three probes from Chapter 2, run as one bash script. The probes are mechanical, ~30 seconds each.

Three probes — one script

```
# probes.sh — runs all three Chapter 2 probes
echo "=== Probe 1: timestamp staleness ==="; python
echo "=== Probe 2: overlap (forward-OOS hold horizo
echo "=== Probe 3: holdout purity ==="; python prob
echo "=== ALL PROBES PASSED ==="
```

The script is one bash command, the orchestrator invokes it once. If any probe fails, the orchestrator's typed packet is FAIL and the morning is over for this question. If all pass, the orchestrator moves to the sub-agent fan-out.

7.3 The 30-minute sub-agent fan-out

ORIENT: *the big picture before the detail*

The fan-out is the expensive step on M3: 200 sub-agents, each evaluating one feature variation. Orchestrator context stays thin; each sub-agent has an isolated context window.

The fan-out is orchestrated by a single Workflow with `parallel()` — the pattern from Chapter 3 of the Loop Engineering workbook. The 200 sub-agents run concurrently (with a 16-concurrent cap from the Workflow tool), each evaluates one variation, returns a typed packet. The orchestrator sees only the 200 typed packets, not the backtest outputs. Wallclock: ~30 minutes (bottlenecked by the 16-concurrent cap).

Cost on M3: 200 sub-agent calls × ~$0.022 per call = ~$4.40 in pure overhead, plus the backtest compute itself

(no LLM cost). The pre-registration cost check from Chapter 1 (~$40 budget) is far from being hit.

7.4 The 5-minute verdict and the 2-minute registry write

ORIENT: *the big picture before the detail*

The verdict is pre-registered: t-stat > 2.5, BSR > 0.85, DSR > 0.50. The orchestrator reads the 200 typed packets and applies the criteria. The output is one verdict: SHIP, ITERATE, or KILL.

SHOW: *worked example (skip if confident)*

The 200 typed packets arrive. The orchestrator runs the verdict: 7 variations clear all three criteria. The pre-registered verdict would be ITERATE (one variation cleared, not the majority). The user pre-registers the additional clause: 'ITERATE if 3+ variations clear; SHIP if 5+'. 7 variations clearing means SHIP.

The registry write is the discipline that makes the research durable: every verdict, every cell, every criterion is recorded in `STRATEGY_REGISTRY.md`. The next morning, the user can pick up where they left off, and the audit log proves what was pre-registered and what was observed.

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. What is the total wallclock of the morning routine?
2. Why is the 5-minute pre-flight the most-leverage minute?
3. What does the 2-minute registry write accomplish?

Answers: *(1) ~55 minutes: 5 (pre-flight) + 10 (probes) + 30 (fan-out) + 5 (verdict) + 2 (registry). (2) a non-question rejected here saves the entire day. (3) the verdict, the cell, and the criteria become durable evidence; the next morning's research can build on them, and the audit log proves what was pre-registered.*

RECAP: *back to altitude*

- Morning routine: 5 + 10 + 30 + 5 + 2 = ~55 minutes total.
- M3 cost: ~$0.40 in input bill for the orchestrator + ~$4.40 for the fan-out.
- The pre-flight is the most-leverage minute; the verdict is the most-leverage verdict.
- The registry write is what makes the research durable.

SECTION GLOSSARY

Pre-flight: the 5-minute G1 sub-agent check; the cheapest gate to fail. **Fan-out:** the 30-minute sub-agent evaluation of N feature variations. **Verdict:** the pre-registered outcome (SHIP / ITERATE / KILL) written to STRATEGY_REGISTRY.md.

Chapter 8. Composing the stack into one routine

BOTTOM LINE ✓ The full agent stack from Chapter 1 is one workflow: pre-flight → probes → fan-out → verdict → registry write. It runs in < 1 hour wallclock on M3 with < $5 in input cost; it is reproducible, auditable, and scales to any research question.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Wire the full morning routine as one Claude Code workflow.
- Identify the failure modes that break the routine under stress.
- Schedule the routine as a CronCreate (session) for repeated research questions.

Prerequisites: Chapter 7.

8.1 The full stack as one workflow

ORIENT: *the big picture before the detail*

The full stack has five layers, each with a typed-packet return. The orchestrator holds the entire stack in a single Workflow.

PRIME: *new terms, one line each*

- **Stack**: the layered agent hierarchy: orchestrator + judgment + assembly + mechanical + data.
- **Typed packet**: the structured return schema that lets each layer pass work to the next without prose parsing.
- **Reproducibility**: the property that running the same workflow on the same data produces the same verdict.

The stack as a single Workflow:

Full morning routine — one Workflow

```
phase('pre-flight')
  g1 = agent('G1 sub-agent', schema=G1_PACKET)
  if g1.verdict != 'PASS': halt()
phase('probes')
  parallel([probe_staleness, probe_overlap, probe_h
  if any_failed: halt()
phase('fan-out')
  pipeline(
    [variation_1, ..., variation_200],
    v => agent(f'eval {v}', schema=IC_PACKET)
  )
phase('verdict')
  v = agent('verdict sub-agent', schema=VERDICT_PACI
phase('registry write')
  write_to_registry(v.verdict, v.criteria, v.cell)
```

The workflow runs in < 1 hour on M3 and the orchestrator's context stays thin throughout: each phase's output is a typed packet (~2K tokens), and the orchestrator only holds the current phase's output plus the cumulative registry write. Total orchestrator context at the end of the morning: ~10K tokens. Total sub-agent context: 200 × ~50K tokens (isolated, freed on completion).

8.2 Failure modes under stress

ORIENT: *the big picture before the detail*

The routine is robust to three common failure modes; it is not robust to a fourth. Knowing which is which saves the morning.

PRIME: *new terms, one line each*

- **Probe failure**: the orchestrator halts the workflow; the user must re-source the data.
- **Fan-out timeout**: a sub-agent takes > 5 minutes; the workflow reschedules it and continues.
- **Verdict ambiguity**: the pre-registered criteria are not all clear (e.g. t-stat > 2.5 but BSR = 0.84); the orchestrator applies the ITERATE clause.
- **Routine-breaking failure**: the pre-registered criteria are unclear (e.g. 'looks promising'); the user re-registers before re-running.

The fourth failure mode is the one that breaks the routine. **If the pre-registered criteria include subjective clauses ('looks promising', 'is a clean find', 'is the kind of edge I would trade'), the verdict sub-agent will rationalise after the fact.** The escape clause from Chapter 3 must be a data-integrity clause, not a vibe clause.

8.3 Scheduling the routine as a CronCreate

ORIENT: *the big picture before the detail*

For repeated research questions (e.g. a daily morning sweep of candidate signals), the routine can be scheduled as a CronCreate session-only job. The job fires at 7am; it runs the pre-flight against any new question submitted the prior evening; the verdict lands in STRATEGY_REGISTRY.md before the user's first coffee.

CronCreate — session-only, 7am daily

```
CronCreate(
  cron='7 7 * * *',
  prompt='Run the morning research routine against
)
```

The CronCreate is the loop-engineering piece that turns a one-off morning routine into a daily research engine. The pattern is from Chapter 3 of the Loop Engineering workbook: a durable primitive with an idempotent terminal-state. The terminal-state is the verdict write; idempotency means re-running the same job produces the same verdict (the pre-registration protects against drift).

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. What is the total orchestrator context at the end of the morning routine?
2. What is the routine-breaking failure mode?
3. How does CronCreate turn a one-off morning into a daily engine?

Answers: *(1) ~10K tokens; each phase's output is a typed packet, and the orchestrator only holds the current phase's output plus the registry write. (2) the pre-registered criteria include subjective clauses ('looks promising'); the verdict sub-agent will rationalise after the fact. (3) by scheduling the routine as a session-only job that fires at 7am daily, with idempotent verdict-write as the terminal state and the pre-registration as the guard against drift.*

RECAP: *back to altitude*

- The full stack is one Workflow: pre-flight → probes → fan-out → verdict → registry.
- Orchestrator context stays ~10K tokens; sub-agent contexts are isolated.
- The routine-breaking failure mode is subjective pre-registration clauses.
- CronCreate turns the one-off morning into a daily research engine.

SECTION GLOSSARY

Workflow: the Claude Code primitive that orchestrates agents + tools via typed-packet flow. **Idempotent terminal state:** a verdict write that, if re-run, produces the same outcome. **Routine-breaking failure:** a pre-registration clause that allows post-hoc rationalisation.

BOTTOM LINE The quant research agent stack on M3 is a five-layer workflow: pre-flight → probes → fan-out → verdict → registry. Each layer is a typed-packet flow. Each gate is pre-registered. Each verdict is durable.

Where this leaves us

You now have a complete quant research stack on M3:

- **Part A** — the agent stack: pre-flight triage, data integrity, backtest design.
- **Part B** — the gates as workflows: G1–G3 as sub-agents, G4–G9 as hooks and monitors, G10 as the exit gate.
- **Part C** — the morning routine: 5 + 10 + 30 + 5 + 2 minutes end-to-end, < $5 in input cost on M3, fully reproducible.

The single rule that survives every chapter: *every verdict is pre-registered, every probe runs every time, every verdict is written to the registry*. Without all three, the stack will eventually lie to you. With all three, it will eventually compound.

Evidence base

- **MODEL_CONTEXT_LOOPS_RESEARCH.md** — the model + context + loops research, updated 2026-07-08 with the M3 probe pass. Part 1 §1.1–1.4 carries the vendor + probe facts cited throughout this book.
- **M3_PROBE_RESULTS.md** — 10 endpoint probes against `https://api.minimax.io/anthropic`: reachability, max output, tool use, structured output, prompt cache (non-functional), vision, rate limits (none), count-tokens, model list, 100K context.
- **METRICS_FRAMEWORK.md** — the 12-metric framework and the 3-metric floor; the gate for whether an improvement is real or noise.
- **code.claude.com/docs/en/** — the Anthropic Claude Code primary docs cited for sub-agents, hooks, skills, and best-practices throughout.
- **Anthropic *Building Effective Agents*** — the workflows-vs-agents stance that justifies the cheapest-reliable-pattern rule.

Reading order

If this is your first quant workflow on M3, read Chapters 1, 2, 4, 6, and 7 in that order. The rest is reference for when the routine breaks (failure-mode diagnosis) or scales (the daily CronCreate).

If you already have a quant workflow and want to retrofit it for M3, start at Chapter 3 (the M3 cost model) and Chapter 5 (the hook layer); the rest is reuse from your existing discipline.

If you are migrating from Claude Sonnet 4.6 to M3, Chapter 3's cost model is the difference that hurts: cache that was amortizing your system prompt is gone. Sub-agents and off-context files are no longer optional.